

PeltOS-Spinlock针对CortexM4的深度优化



PeltOS札记。中文名兽皮的这款裸机操作系统在网上直播创作过程。之所以取名兽皮是因为总比裸机好点，等那一天渔夫的能耐见长，再整点布衣穿穿。老百姓过日子，柴米油盐，写程序纯粹是喜欢，没啥宏大叙事。

小时候渔夫用机器指令直接手写程序，练就代码优化的底子。对于操作系统的基础元件，哪怕有丝毫的优化，效果也是可观。

还是新唐Nuvoton的M463处理器，CortexM4架构。上一版：

```
/* Raw atomic try-acquire: return 1 if success, 0 if busy */
static inline int32_t __piSpinlockRaw(volatile int32_t* addr) {
    uint32_t old_val;
    uint32_t new_val;
    uint32_t status;
    do {
        __asm volatile(
            "ldrex %0, [%3]      \n\t" /* old_val = *addr */
            "cmp    %0, #0      \n\t" /* check if unlocked */
            "bne    1f          \n\t" /* if locked, jump to fail */
            "mov    %1, #1      \n\t" /* new_val = 1 */
            "strex %2, %1, [%3]  \n\t" /* try store */
            "b      2f          \n\t"
            "1:              \n\t"
            "mov    %2, #2      \n\t" /* mark as busy */
            "2:              \n\t"
            : "=&r"(old_val), "=&r"(new_val), "=&r"(status)
            : "r"(addr)
            : "cc", "memory"
        );
        if (status == 2) {
            return 0; /* already locked */
        }
    } while (status != 0);
    return 1;
}
```

这是一版RAW版的半成品自旋锁：如果之前已经锁上了返回0，如果锁空闲，锁上，然后返回1。

这个半成品最好的用处是在全异步环境下作为某些代码的防重入标记。组装上死等循环就是自旋锁。

自媒体写东西就是这样，这些上一节已经讲过了，这次看帖子的看官未必看过上一帖。

这一次渔夫要针对新唐Nuvoton的M463处理器做一版深度优化。其实也适合STM32H7已经STM32F4等等。

有看官可能觉得这段程序已经可以啦，还要怎么优化。

```
static inline uint32_t __pilsLocked(uint32_t* sl) {
    uint32_t tmp, status;
    __asm__ volatile (
        /* 1. Data Memory Barrier ensures preceding memory accesses complete */
        "    dmb ish          \n\t"
        "1: ldrex %0, [%2] \n\t" /* Load lock status */
        "    mov  %1, #1      \n\t" /* locked status for this attempt */
        "    cmp  %0, #0      \n\t" /* Check if lock is free */
        "    ite  eq          \n\t" /* IT block for conditional execution */
        "    strex %1, %1, [%2] \n\t" /* Attempt to acquire */
        "    movne %1, #2      \n\t" /* Set failure status if already locked */
        "    cmp  %1, #1      \n\t" /* Check strex result (1 = acquisition failed) */
        "    beq  1b          \n\t" /* Retry if lock acquisition failed */
        /* 2. DMB prevents reordering of critical section memory access */
        "    dmb ish          \n\t"
        : "=&r"(tmp), "=&r"(status)
        : "r"(sl)
        : "cc", "memory"
    );
    return status; // 0 if lock acquired, 2 if busy
}
```

开始的这一句dmb ish告诉处理器，清场子啦。在这条指令以前的内存操作全部完成，别跟我后面的混起来。

还是用LDREX指令读锁的状态并开启Exclusive Monitor。

mov %1,#1是把1这个立即数放在status里面，其实这里玩了一个障眼法，这个#1其实是要写入锁的值，对比上一版叫new_val。

%1是从内存读出来的锁的值，tmp，上一版叫old_val，cmp %1,#0，看看现在锁是不是锁住？等于0是没锁，不等于0是锁住。

精彩的这一句在这里 ite eq，strex和movne,这三条指令组成一个小组。后面这两条是一起放在流水线上的，根据上一条指令的标志位，如果等于#0执行strex这一条，如果不等于零走mov这一条。前者尝试更新锁的值，上锁，如果成功%1等于0，不成功等于1；后者直接给%1也就是status赋值#2，表示进来时锁已经被锁上了。

到cmp %1, #1这里三选一，写入不成功再试，也就是 beq 1b。

省去了上一版的跳转指令，流水线畅通无比，程序效率高多了。

最后阶段评估代码，发现了一点瑕疵。LDREX与STREX是配对执行的，当执

行LDREX时启动独占监视器Exclusive Monitor，STREX结束这次监视。而当锁已经锁住的情况下，没有STREX就退出啦，这次监视活动要等到下次LDREX才算结束并重新开始。问题并不是很大，渔夫也想不大具体有多大的危害。总之让人觉得别扭。ARM官方建议用CLREX指令显式的终止监视。

这就需要修改。可是流水线的两边，一边一条，现在NE这边变成两条，mov %1,#2和clrex，而那边只有一条STREX。其实，EQ这边也有两条，当时为了对付NE的一条，挪到前面去了。现在拿回来，正好每一边两条。

```
static inline uint32_t __pilsLocked(uint32_t* sl) {
    uint32_t tmp, status;
    __asm__ volatile (
        /* 1. Data Memory Barrier ensures preceding memory accesses complete */
        "    dmb ish        \n\t"
        "1: ldrex %0, [%2] \n\t" /* Load lock status */
        "    cmp    %0, #0    \n\t" /* Check if lock is free */
        "    itete eq        \n\t" // IT block: EQ, NE, NE
        "    movseq %1, #1    \n\t" // EQ: Set status to 1 (prepare to acquire)
        "    movsne %1, #2    \n\t" // NE: Set status to 2 (lock is busy)
        "    strexeq %1, %1, [%2] \n\t" // EQ: Attempt to acquire the lock
        "    clrexne        \n\t" // NE: Clear exclusive monitor
        "    cmp    %1, #1    \n\t" /* Check strex result (1 = acquisition failed) */
        "    beq    1b        \n\t" /* Retry if lock acquisition failed */
        /* 2. DMB prevents reordering of critical section memory access */
        "    dmb ish        \n\t"
        : "=&r"(tmp), "=&r"(status)
        : "r"(sl)
        : "cc", "memory"
    );
    return status; // 0 if lock acquired, 2 if busy
}
```

流水线组队记号itete eq，如果%0==0时执行movsbeq %1, #1，然后，strexeq %1,%1,[%2]，更新锁状态为锁住。否则执行movsbne %1,#2，然后，CLREX清除本次独占监视器。

由于两边指令很平衡，渔夫专门用movs指令来优化一下，对于小于256的立即数，movs指令比mov的代码短，且耗时更少。一下子回到了40几年前机器码手写编程的时代，好亲切呀，久违了。

而且，199x年前后那些年，给很多届硕士生讲授《计算机体系结构》，那个时候就讲流水线预测。备课的时候就想，怎么样让预测和流水线更好的匹配？时隔30年，终于亲自体验了一把具体的流水线排列操作。

现代编译器的高性能，基本上没机会去考虑流水线的编排这类事情，也就是写操作系统内核这样机会才可能碰到，痛快。

渔夫60多岁还在写程序，图的就是开心，这样级别的程序优化，绝大多数人一辈子都遇不到。优化成功的渔夫收获了不少内啡肽，无比的享受。

兽皮的某些内容可以在<https://github.com/Petereetic>上看到。

愿意与渔夫切磋的看官，微信加PeterInnoStudio